

PrivacyGNN Beginner Full Guide

A slow, plain-English explanation of the entire project

Prepared for Krish Sharma

May 11, 2026

Abstract

This guide explains the PrivacyGNN project from the ground up. It assumes you are new to machine learning privacy, graph neural networks, experiment code, and research result files. The goal is not just to say what each file does, but to explain why it exists, what problem it solves, how the pieces connect, and what you should think while reading the code. It intentionally repeats ideas in different ways because beginners learn best when the same idea is seen from multiple angles.

Contents

1	How to Use This Guide	2
2	The Project in One Sentence	2
3	A Beginner Analogy	3
4	What Is a Graph?	3
4.1	Tiny Example	4
5	What Is Node Classification?	4
6	What Is Machine Learning Privacy Leakage?	4
6.1	Member and Non-Member	5
6.2	Why Confidence Matters	5
7	What Is a Graph Neural Network?	5
7.1	Why This Could Affect Privacy	6
8	The Four Research Questions	6
8.1	RQ1: Are GNNs More Vulnerable Than Non-Graph Models?	6
8.2	RQ2: How Do Homophily and Density Affect Leakage?	6
8.3	RQ3: Can Lightweight Defenses Reduce Leakage?	6
8.4	RQ4: What Is the Utility-Privacy Tradeoff?	6

9 The Whole Project Workflow	7
10 The Repository Map in Plain English	7
11 Configuration: The Experiment Menu	8
11.1 What Is a Seed?	8
11.2 Why So Many Runs?	8
11.3 Why LogReg and MLP Skip Defenses	8
12 Data Generation Explained Slowly	9
12.1 Step 1: Assign Labels	9
12.2 Step 2: Create Class Centers	9
12.3 Step 3: Create Node Features	9
12.4 Step 4: Decide How Many Edges	9
12.5 Step 5: Decide Same-Label vs Different-Label Edges	9
12.6 Step 6: Create Train/Test Masks	10
13 Real Datasets: Cora and Citeseer	10
14 Models Explained Like a Beginner	10
14.1 Logistic Regression	10
14.2 MLP	10
14.3 GCN	11
14.4 GraphSAGE	11
15 Training Explained	11
15.1 The Training Loop	11
15.2 What Is Loss?	12
15.3 What Is Dropout?	12
16 Defenses Explained Slowly	12
16.1 No Defense	12
16.2 DropEdge	12
16.3 Label Smoothing	12
16.4 Early Stopping	13
16.5 Confidence Masking	13
16.6 Edge Sparsification	13
16.7 DP-SGD	13

17 Membership Attacks Explained Slowly	14
17.1 The Four Attack Features	14
17.2 Confidence Attack	14
17.3 Threshold Attack	14
17.4 Shadow Attack	14
17.5 LiRA	15
18 Metrics Explained	15
18.1 Accuracy	15
18.2 F1 Score	15
18.3 Task AUROC	15
18.4 Attack AUC	15
18.5 ECE	16
19 The Most Important File: experiment.py	16
19.1 What run_one Does	16
19.2 Why It Returns a Dictionary	17
20 The Main Runner	17
20.1 Simple Mental Model	17
20.2 What It Saves	17
21 How to Read all_results.csv	17
22 How to Read summary.csv	18
22.1 Mean and Standard Deviation	18
23 Statistics Files Explained	18
23.1 significance.csv	18
23.2 How to Interpret diff	19
23.3 summary_bootstrap.csv	19
24 Figure Scripts Explained	19
24.1 Why This Matters	19
24.2 Main Figure Scripts	20
25 Paper and Slide Assets	20
26 Current Limitations Explained Simply	20

26.1 LiRA Is Not Really Implemented	20
26.2 Large Graphs Are Not Really Implemented	20
26.3 DP-SGD Is Not Really Active	20
26.4 No Automated Tests	20
27 What Tests Should Exist	21
28 How to Explain the Project Out Loud	21
29 A Study Plan for Actually Learning It	21
29.1 Day 1: Understand the Story	21
29.2 Day 2: Understand the Pipeline	22
29.3 Day 3: Understand Data and Models	22
29.4 Day 4: Understand Attacks	22
29.5 Day 5: Understand Results	22
29.6 Day 6: Understand Figures and Paper	22
30 Beginner Glossary	23
31 Final Beginner Mental Model	23

1 How to Use This Guide

This is a beginner-first guide. That means it moves slowly. If something feels obvious, you can skim it. If something feels confusing, pause and reread the example. The goal is that by the end you can explain your project to another person without just memorizing file names.

There are three levels of understanding:

1. **Story level:** What question is the project trying to answer?
2. **Experiment level:** What is actually run, measured, saved, and plotted?
3. **Code level:** Which file/function performs each step?

Most confusion happens when these levels get mixed together. For example, “membership inference attack” is a research idea, but `confidence_attack` in `attacks.py` is the actual code implementation of one attack. This guide keeps those apart.

Recommended reading plan:

- First pass: read Sections 1–8 to understand the big ideas.
- Second pass: read Sections 9–18 with the code open beside you.
- Third pass: use Sections 19–25 as your reference when running experiments or explaining results.

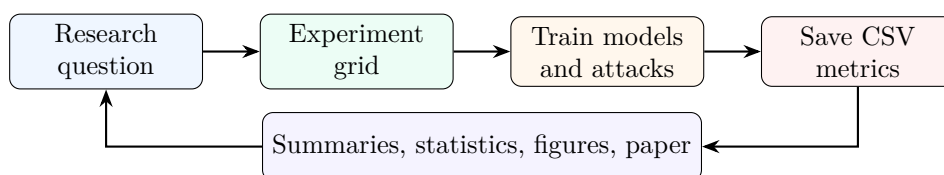
2 The Project in One Sentence

PrivacyGNN studies whether graph neural networks reveal which nodes were used during training, and whether simple defenses can reduce that leakage without hurting accuracy too much.

That sentence has many parts:

- **Graph neural networks:** machine learning models that learn from points connected by edges.
- **Reveal which nodes were used during training:** privacy leakage called membership inference.
- **Simple defenses:** practical tricks like dropping edges, smoothing labels, masking confidence scores, or stopping early.
- **Without hurting accuracy:** the privacy-utility tradeoff.

The project is not just one model. It is a whole experiment system:



3 A Beginner Analogy

Imagine a teacher trains for an exam using a private study list of students. Later, someone asks the teacher questions about many students and watches how confident the teacher sounds.

If the teacher sounds extremely confident about certain students and less confident about others, an attacker may guess: “The very confident ones were probably on the private study list.”

That is the core idea of a membership inference attack.

Now replace the teacher with a machine learning model:

- The **private study list** is the training set.
- The **students** are graph nodes.
- The **teacher’s confidence** is the model’s predicted probability.
- The **attacker’s guess** is member vs non-member.

The project asks: does this guessing work better for graph neural networks? And can we make it harder?

4 What Is a Graph?

A graph is a set of objects and relationships.

- Objects are called **nodes**.
- Relationships are called **edges**.

Examples:

- In a social network, people are nodes and friendships are edges.
- In a citation network, papers are nodes and citations are edges.
- In a medical graph, patients or conditions can be nodes and relationships can be edges.

In code, a graph is usually represented by:

- **x**: node features, meaning numbers that describe each node.
- **edge_index**: pairs of node IDs that say which nodes are connected.
- **y**: labels, meaning the correct class for each node.
- **train_mask**: which nodes are used for training.
- **test_mask**: which nodes are held out for testing.

In PyTorch Geometric, these live in a **Data** object.

4.1 Tiny Example

Suppose there are four papers:

- Paper 0: Machine Learning
- Paper 1: Machine Learning
- Paper 2: Biology
- Paper 3: Biology

If Paper 0 cites Paper 1, and Paper 2 cites Paper 3, then the graph has edges:

$$(0, 1), (1, 0), (2, 3), (3, 2)$$

The project uses undirected-style edges by adding both directions.

5 What Is Node Classification?

Node classification means predicting the label of a node.

In this project, every node has a label like:

$$y_i \in \{0, 1, 2, 3, 4\}$$

The model receives:

- the node's feature vector,
- possibly the graph connections,
- labels for training nodes only.

The model outputs probabilities:

$$[0.05, 0.10, 0.75, 0.04, 0.06]$$

This means the model thinks class 2 is most likely.

Accuracy asks: did the model pick the correct class?

Privacy attack AUC asks: can an attacker use these probabilities to guess whether this node was in training?

Those are different questions.

6 What Is Machine Learning Privacy Leakage?

Many people think privacy is safe if the raw data is hidden. But machine learning models can leak information through their outputs.

Example:

- The model never shows its training set.
- The model only shows prediction probabilities.
- But those probabilities may be unusually confident for training examples.
- An attacker can exploit this pattern.

This is why membership inference matters.

6.1 Member and Non-Member

In this project:

- **Member** means the node was in the model's training set.
- **Non-member** means the node was not in training, usually in the test set.

The attacker tries to classify:

node $\rightarrow$ member or non-member

6.2 Why Confidence Matters

Models often behave differently on examples they trained on:

- They may be more confident.
- They may have lower entropy.
- They may put more probability on the true class.

These differences are exactly what the attacks in `attacks.py` measure.

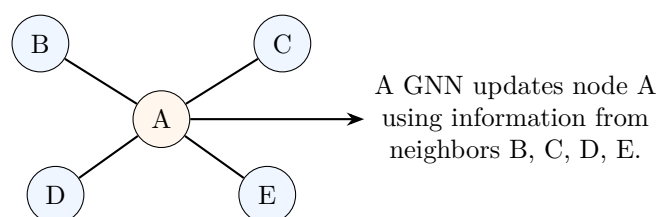
7 What Is a Graph Neural Network?

A normal model like logistic regression or an MLP looks at one node's features by itself.

A graph neural network also looks at neighbors.

The simple idea:

To understand a node, look at the node and the nodes connected to it.



7.1 Why This Could Affect Privacy

Graph learning can affect privacy in two opposite ways:

- It might **increase leakage** because a node's training information can spread through edges.
- It might **reduce leakage** because neighbor averaging can smooth predictions and reduce memorization.

Your project studies which actually happens under different graph structures.

8 The Four Research Questions

The README gives four research questions. Here they are in beginner language.

8.1 RQ1: Are GNNs More Vulnerable Than Non-Graph Models?

This compares:

- LogReg and MLP: models that ignore edges.
- GCN and GraphSAGE: models that use edges.

The question is whether using graph structure makes membership attacks easier.

8.2 RQ2: How Do Homophily and Density Affect Leakage?

Homophily means similar nodes connect to each other.

Density means how many edges the graph has.

The project creates synthetic graphs where these can be changed separately:

high/low homophily $\times$ sparse/medium/dense density

8.3 RQ3: Can Lightweight Defenses Reduce Leakage?

A defense is a change meant to make membership inference harder. The project tests simple defenses that are easy to run.

8.4 RQ4: What Is the Utility-Privacy Tradeoff?

Utility means the model is useful, usually high accuracy.

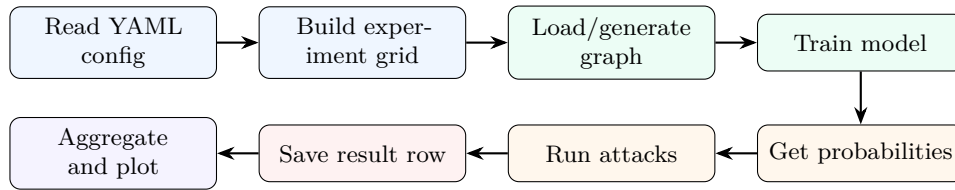
Privacy means the attack does poorly, usually attack AUC near 0.5.

The best situation is:

high accuracy and attack AUC near 0.5.

9 The Whole Project Workflow

The full project workflow looks like this:



Each row in `results/all_results.csv` comes from one pass through this workflow.

10 The Repository Map in Plain English

File	Beginner explanation
<code>config.py</code>	Decides what experiments exist. It reads the YAML file and expands it into many combinations.
<code>experiment_config.yaml</code>	The main menu of experiment options: datasets, models, defenses, seeds, attacks.
<code>experiment_config_paper.yaml</code>	A smaller, cleaner menu for the paper-style experiment grid.
<code>data.py</code>	Creates or loads graph data. Also measures homophily and density.
<code>models.py</code>	Defines the two GNN model classes: GCN and GraphSAGE.
<code>training.py</code>	Teaches a GNN to classify nodes. Includes defense options.
<code>attacks.py</code>	Tries to guess train membership from model probabilities.
<code>experiment.py</code>	Runs one complete experiment: data, model, defense, attack, metrics.
<code>run_final.py</code>	Runs all experiments in the grid and writes CSV files.
<code>stats_utils.py</code>	Compares defenses statistically and computes bootstrap confidence intervals.
<code>generate_figures.py</code>	Makes paper-style figures from saved results.
<code>generate_basic_figures.py</code>	Makes simpler figures for understanding synthetic results.
<code>generate_final_visuals.py</code>	Makes polished final visuals for presentation/poster use.
<code>generate_poster_visuals.py</code>	Makes a larger poster visual suite.
<code>ogb_loader.py</code>	Placeholder for large graph datasets. Not active in this checkout.
<code>graph_minibatch.py</code>	Placeholder for minibatch training and DP-SGD. Not active in this checkout.
<code>lira_attack.py</code>	Placeholder for LiRA attack. Currently returns random-guess values.
<code>results/*.csv</code>	The actual experiment outputs. These are the source of truth.
<code>paper/</code>	Paper, manuscript, slide, and literature assets.

11 Configuration: The Experiment Menu

The project is config-driven. This means you do not hard-code every experiment in Python. Instead, you write choices in YAML.

Think of `experiment_config_paper.yaml` like a restaurant menu:

- datasets,
- models,
- attacks,
- defenses,
- training settings,
- seeds.

Then `config.py` combines them.

11.1 What Is a Seed?

A seed is a number that controls randomness. If you use the same seed, you should get the same random split or synthetic graph. If you use multiple seeds, you test whether results are stable.

Your paper config uses:

42, 123, 456, 789, 1024

11.2 Why So Many Runs?

If you have:

- 8 datasets,
- 4 models,
- multiple defenses,
- 5 seeds,

then one project becomes hundreds of experiments. That is why `run_final.py` exists.

11.3 Why LogReg and MLP Skip Defenses

Many defenses are graph-specific:

- DropEdge needs edges.
- Edge sparsification needs edges.

LogReg and MLP ignore edges, so the code only runs them with `none`.

12 Data Generation Explained Slowly

The most important function in `data.py` is `make_synthetic`.

It creates a fake graph where the project controls two knobs:

- homophily,
- density.

12.1 Step 1: Assign Labels

Each node gets a class label from 0 to 4. If there are 400 nodes and 5 classes, the labels might be spread across those classes.

12.2 Step 2: Create Class Centers

For each class, the code creates a feature center. Nodes in the same class are generated near the same center. This makes classification possible.

Beginner analogy:

Each class has a home base in feature space. Nodes from that class are placed near that home base.

12.3 Step 3: Create Node Features

For each node:

$$\text{node feature} = \text{class center} + \text{noise}$$

Noise makes the task realistic. Without noise, the classes would be too perfect.

12.4 Step 4: Decide How Many Edges

Density controls the target number of edges:

- sparse: few edges,
- medium: more edges,
- dense: many edges.

12.5 Step 5: Decide Same-Label vs Different-Label Edges

Homophily controls edge type:

- high homophily: many edges connect same-label nodes,
- low homophily: many edges connect different-label nodes.

12.6 Step 6: Create Train/Test Masks

The code randomly splits nodes:

- first half: training members,
- second half: test non-members.

This is what makes membership inference measurable.

13 Real Datasets: Cora and Citeseer

Cora and Citeseer are citation networks.

In a citation network:

- nodes are papers,
- edges are citation links,
- features describe paper content,
- labels are paper categories.

`load_citation` uses PyTorch Geometric's Planetoid loader. If the normal download URL fails, the code tries a mirror.

Important beginner note: real datasets are messier than synthetic datasets. Synthetic datasets let you control variables. Real datasets show whether the story holds outside a controlled lab setting.

14 Models Explained Like a Beginner

14.1 Logistic Regression

Logistic regression is the simplest baseline. It draws a boundary between classes using the node's features.

It does not know the graph exists.

Why include it?

- It shows how much can be learned from features alone.
- It is a low-complexity comparison point.

14.2 MLP

MLP stands for multilayer perceptron. It is a small neural network. In this project, it uses hidden layers:

$$64 \rightarrow 32$$

It also ignores the graph. It can learn nonlinear feature patterns but cannot use neighbors.

14.3 GCN

GCN stands for Graph Convolutional Network.

It updates a node by mixing information from neighbors. A simple mental model:

A GCN says: “I will classify this node after looking at its own features and nearby nodes.”

The project’s GCN has:

- first graph convolution,
- ReLU activation,
- dropout,
- second graph convolution.

14.4 GraphSAGE

GraphSAGE is another GNN. It also uses neighbors, but its aggregation method is different from GCN.

Beginner mental model:

GCN and GraphSAGE are two different ways to summarize a node’s neighborhood.

Comparing them helps answer whether privacy leakage is caused by all graph models or by specific architectures.

15 Training Explained

Training means changing model weights so predictions become more correct on training nodes.

15.1 The Training Loop

`train_gnn` does this repeatedly:

1. Clear old gradients.
2. Run the model forward.
3. Compare predictions to true labels.
4. Compute loss.
5. Backpropagate.
6. Update weights.

15.2 What Is Loss?

Loss is a number measuring how wrong the model is. Lower loss means better training predictions. The normal loss is cross-entropy:

$$\text{loss} = -\log(\text{probability assigned to true class})$$

If the true class gets high probability, loss is small. If the true class gets low probability, loss is large.

15.3 What Is Dropout?

The GNN models use dropout. Dropout randomly hides some internal values during training. It helps reduce memorization.

Memorization matters because membership attacks often exploit memorization.

16 Defenses Explained Slowly

A defense tries to reduce leakage. The project tests lightweight defenses. Lightweight means they are easier to use than heavy formal privacy methods.

16.1 No Defense

`none` is the baseline. You always need a baseline because you need to know whether a defense actually improves anything.

16.2 DropEdge

DropEdge randomly removes edges during training.

Analogy:

If the graph is a road map, DropEdge temporarily closes random roads each training epoch.

Why might this help?

- It may stop the model from relying too heavily on exact graph structure.
- It may reduce overfitting.

16.3 Label Smoothing

Normally, the true label is treated as 100 percent correct:

$$[0, 0, 1, 0, 0]$$

Label smoothing makes it softer:

[0.025, 0.025, 0.90, 0.025, 0.025]

Why might this help?

- It can reduce overconfidence.
- Less overconfidence might make membership attacks harder.

But your results suggest label smoothing can sometimes increase leakage. That is important: a defense that sounds good can fail in practice.

16.4 Early Stopping

Early stopping ends training before the model overfits too much.

Analogy:

Stop studying before you memorize the answer key instead of learning the topic.

16.5 Confidence Masking

Confidence masking changes the output probabilities after training. It keeps only the top k probabilities and sets the rest to zero.

Example:

[0.70, 0.20, 0.05, 0.03, 0.02]

With top-2 masking:

[0.70, 0.20, 0, 0, 0] $\rightarrow$ [0.777, 0.222, 0, 0, 0]

Why might this help?

- It hides small probability details that attackers might use.

16.6 Edge Sparsification

Edge sparsification removes some edges before training.

It is different from DropEdge:

- DropEdge removes random edges temporarily each epoch.
- Edge sparsification removes edges once before training.

16.7 DP-SGD

The config mentions DP-SGD, which stands for Differentially Private Stochastic Gradient Descent. But in this checkout, the necessary minibatch training function is a stub. So DP-SGD is not actually usable here yet.

17 Membership Attacks Explained Slowly

All attacks start from model probabilities.

Suppose a model outputs:

[0.01, 0.02, 0.94, 0.02, 0.01]

This output is very confident. The attacker may suspect this node was in training.

17.1 The Four Attack Features

`extract_features` turns each probability vector into four numbers:

1. **Max probability:** how confident is the model in its top guess?
2. **True-label probability:** how much probability did it give the correct class?
3. **Entropy:** how spread out is the probability distribution?
4. **Margin-like confidence feature:** another confidence summary.

Beginner intuition:

The attack does not look at the entire probability vector directly. It summarizes the vector into confidence clues.

17.2 Confidence Attack

The confidence attack trains a small logistic regression attack model.

It asks:

Given these four confidence clues, does this look like a member or a non-member?

17.3 Threshold Attack

The threshold attack is simpler. It says:

If true-label confidence is high enough, guess member. Otherwise guess non-member.

17.4 Shadow Attack

The shadow attack trains another model to imitate the target situation.

Analogy:

If you cannot see how the target model learned, train your own similar model and study its behavior.

Steps:

1. Train a shadow model.
2. Observe shadow members and non-members.
3. Train an attacker on those examples.
4. Use that attacker on the target model.

17.5 LiRA

LiRA is a stronger modern attack idea, but the current project file `lira_attack.py` is only a placeholder. It returns 0.5, which means random guessing.

So do not interpret current LiRA numbers as real LiRA results.

18 Metrics Explained

18.1 Accuracy

Accuracy is:

$$\frac{\text{correct predictions}}{\text{total predictions}}$$

High accuracy means the model is useful for classification.

18.2 F1 Score

F1 balances precision and recall. Macro F1 averages across classes, which helps when classes are imbalanced.

18.3 Task AUROC

This measures classification quality in a probability-aware way. For multiclass data, the project uses one-vs-rest averaging.

18.4 Attack AUC

Attack AUC measures membership inference success.

- 0.5: random guessing.
- 0.6: weak but real leakage.
- 0.8: strong leakage.
- 1.0: perfect membership detection.

18.5 ECE

ECE stands for Expected Calibration Error.

Calibration asks:

When the model says it is 80 percent confident, is it correct about 80 percent of the time?

Bad calibration can give attackers useful confidence patterns.

19 The Most Important File: `experiment.py`

`experiment.py` is the core of the project because it runs one complete experiment.

One experiment means:

one dataset + one model + one defense + one seed

19.1 What `run_one` Does

`run_one` does all of this:

1. Load config.
2. Load the dataset.
3. Compute graph homophily and density.
4. Set random seeds.
5. Prepare defense settings.
6. Train the target model.
7. Get prediction probabilities.
8. Apply confidence masking if needed.
9. Compute accuracy, F1, and task AUROC.
10. Run confidence and threshold attacks.
11. Train shadow models if needed.
12. Run shadow attack if needed.
13. Run LiRA placeholder if enabled.
14. Compute ECE.
15. Return one dictionary of metrics.

19.2 Why It Returns a Dictionary

A dictionary is easy to turn into a CSV row.

Example conceptually:

```
{
  "dataset": "Cora",
  "model": "GCN",
  "defense": "dropedge",
  "seed": 42,
  "test_accuracy": 0.87,
  "conf_attack_auc": 0.55
}
```

After many runs, all dictionaries become `all_results.csv`.

20 The Main Runner

`run_final.py` is the file you run when you want the whole experiment grid.

It does not know every detail of training. Instead, it repeatedly calls `run_one`.

20.1 Simple Mental Model

`run_one` makes one result row. `run_final.py` makes all result rows.

20.2 What It Saves

- `all_results.csv`: every run.
- `summary.csv`: averages over seeds.
- `significance.csv`: paired t-tests.
- `significance_lira.csv`: LiRA comparison file, but LiRA is placeholder here.
- `summary_bootstrap.csv`: bootstrap uncertainty intervals.

21 How to Read `all_results.csv`

`all_results.csv` is the raw source of truth.

Each row answers:

What happened for this dataset, model, defense, and seed?

Column	Beginner meaning
dataset	Which graph was used.

Column	Beginner meaning
<code>model</code>	Which model was trained.
<code>defense</code>	Which defense was used.
<code>seed</code>	Which randomness setting was used.
<code>homophily</code>	Fraction of edges connecting same-label nodes.
<code>density</code>	How crowded the graph is with edges.
<code>test_accuracy</code>	How often the model predicted correctly on test nodes.
<code>test_f1</code>	Balanced class performance metric.
<code>test_auroc</code>	Probability-aware classification performance.
<code>conf_attack_auc</code>	How well the confidence attack guessed membership.
<code>threshold_attack_auc</code>	How well the threshold attack guessed membership.
<code>shadow_attack_auc</code>	How well the shadow attack guessed membership.
<code>lira_attack_auc</code>	Placeholder in this checkout; usually 0.5.
<code>ece_test</code>	Calibration error on the test set.
<code>dp_epsilon</code>	DP privacy budget if DP-SGD ran; NaN here for normal runs.

22 How to Read `summary.csv`

`summary.csv` averages over seeds.

If `all_results.csv` has five rows for:

Cora + GCN + none

then `summary.csv` has one averaged row for that combination.

This is useful because one seed can be noisy. Averages give a more stable picture.

22.1 Mean and Standard Deviation

Mean is the average. Standard deviation is the spread.

Example:

- Mean attack AUC = 0.55 means average leakage is modest.
- Std = 0.02 means seeds were fairly consistent.
- Std = 0.10 would mean seeds varied a lot.

23 Statistics Files Explained

23.1 `significance.csv`

This file compares each defense against no defense.

It asks:

For the same dataset, model, and seeds, did this defense change attack AUC?

The key columns:

- `mean_none`: average attack AUC without defense.
- `mean_defense`: average attack AUC with defense.
- `diff`: defense minus none.
- `p_value`: statistical evidence for a difference.

23.2 How to Interpret diff

$$\text{diff} = \text{defense AUC} - \text{no-defense AUC}$$

- Negative diff: defense reduced leakage.
- Positive diff: defense increased leakage.
- Near zero: little change.

23.3 `summary_bootstrap.csv`

Bootstrap means resampling seeds to estimate uncertainty.

Beginner mental model:

If we reran the project with similar seeds, how much might the average move?

24 Figure Scripts Explained

The figure scripts do not train models. They read CSVs and make visuals.

24.1 Why This Matters

If a figure looks wrong, do not immediately blame training. First check:

- Which CSV did the script read?
- Which rows did it filter?
- Which metric did it plot?
- Did the needed dataset/model/defense rows exist?

24.2 Main Figure Scripts

- `generate_figures.py`: paper-style figures.
- `generate_basic_figures.py`: simpler learning figures.
- `generate_final_visuals.py`: polished final visuals.
- `generate_poster_visuals.py`: poster-ready visual suite.
- `fix_figures.py`: improved versions of selected paper figures.

25 Paper and Slide Assets

`paper/ieee_privacy_gnn.tex` is the main LaTeX paper.

`paper/build_manuscript.py` creates a manuscript PDF using ReportLab. Be careful: some text inside that script may be stale compared to the current code.

`paper/build_project_slides.py` creates a PowerPoint deck.

Beginner warning: generated paper text is not automatically guaranteed to match the latest experiment code. Always check claims against:

- `experiment_config_paper.yaml`,
- `run_final.py`,
- `results/all_results.csv`,
- `results/summary.csv`.

26 Current Limitations Explained Simply

26.1 LiRA Is Not Really Implemented

The file exists, but it returns random-guess values. So the project has a place for LiRA, but not a full LiRA implementation.

26.2 Large Graphs Are Not Really Implemented

`ogb_loader.py` and `graph_minibatch.py` are placeholders. This means the current project works for full-batch Cora, Citeseer, and synthetic graphs, but not true large-graph neighbor sampling.

26.3 DP-SGD Is Not Really Active

The config mentions DP-SGD, but the code path depends on minibatch DP training, which is stubbed.

26.4 No Automated Tests

There is no dedicated test suite. The project relies on running experiments and checking results. That is acceptable for a research prototype, but a stronger software project would add unit tests.

27 What Tests Should Exist

If you want to improve the project, add tests like:

- Does `make_synthetic` return the right number of nodes?
- Do train/test masks split nodes correctly?
- Does high homophily actually produce higher homophily than low homophily?
- Does `extract_features` return shape $n \times 4$?
- Does `confidence_attack` return AUC between 0 and 1?
- Does `get_experiment_list` skip invalid LogReg/MLP defense combinations?
- Can `run_one` finish on one tiny synthetic config?

28 How to Explain the Project Out Loud

Here is a beginner-friendly explanation you can use:

My project studies privacy leakage in graph neural networks. The privacy risk is membership inference: an attacker tries to guess whether a node was part of the model's training set by looking at prediction probabilities. I compare graph models like GCN and GraphSAGE against feature-only baselines like logistic regression and MLP. I also create synthetic graphs where I control homophily and density, so I can study how graph structure affects leakage. Then I test simple defenses, such as DropEdge and edge sparsification, and measure whether they reduce attack AUC while preserving accuracy.

If someone asks what the code does:

The code is config-driven. A YAML file defines datasets, models, defenses, attacks, and seeds. The runner expands this into many experiments. Each experiment trains one model, runs membership attacks, computes metrics, and writes one row to a CSV. Other scripts aggregate the CSVs, compute statistics, and generate figures.

29 A Study Plan for Actually Learning It

29.1 Day 1: Understand the Story

Read:

- this guide through Section 8,
- `README.md`,
- `RESULTS_ANALYSIS.md`.

Goal: explain the project without code.

29.2 Day 2: Understand the Pipeline

Read:

- `experiment_config_paper.yaml`,
- `config.py`,
- `run_final.py`.

Goal: explain how many experiments are run and why.

29.3 Day 3: Understand Data and Models

Read:

- `data.py`,
- `models.py`,
- `training.py`.

Goal: explain how synthetic graphs are generated and how GNNs train.

29.4 Day 4: Understand Attacks

Read:

- `attacks.py`,
- attack-related parts of `experiment.py`.

Goal: explain what attack AUC means.

29.5 Day 5: Understand Results

Open:

- `results/all_results.csv`,
- `results/summary.csv`,
- `results/significance.csv`.

Goal: trace one result row back to code.

29.6 Day 6: Understand Figures and Paper

Read:

- `generate_figures.py`,
- `paper/ieee_privacy_gnn.tex`.

Goal: connect claims to metrics and figures.

30 Beginner Glossary

Term	Meaning
Node	One object in a graph, such as a paper or person.
Edge	A connection between two nodes.
Feature	A numeric description of a node.
Label	The correct class for a node.
Train set	Nodes the model learns from.
Test set	Nodes held out to evaluate generalization.
Member	A node that was in training.
Non-member	A node that was not in training.
GNN	A neural network that uses graph edges.
GCN	A graph convolutional network.
GraphSAGE	A GNN architecture with neighbor aggregation.
Homophily	How often connected nodes share the same label.
Density	How many edges exist compared to possible edges.
Defense	A method meant to reduce privacy leakage.
Attack AUC	How well an attack distinguishes members from non-members.
ECE	Expected Calibration Error, a confidence reliability metric.
Seed	A number controlling randomness.
CSV	A spreadsheet-like file storing experiment results.
YAML	A human-readable config file format.

31 Final Beginner Mental Model

The project is easier if you remember this chain:

Config → Data → Model → Predictions → Attack → Results → Figures

Everything in the repository fits somewhere in that chain.

- If you are confused about what runs, look at config.
- If you are confused about graph structure, look at data.
- If you are confused about learning, look at models and training.
- If you are confused about privacy, look at attacks.
- If you are confused about conclusions, look at results and figures.

The main idea: the project measures whether model confidence reveals training membership, and how graph structure plus defenses change that risk.